Complete RTL-to-GDSII Command, TCL Script, and Debugging Guide

Purpose of this Document

This document is a complete practical workflow guide for performing RTL-to-GDSII ASIC implementation using Cadence Genus and Innovus.

The guide includes:

- working commands
- working TCL scripts
- exact execution order
- common errors encountered
- debugging methods
- command explanations
- parameter explanations
- routing fixes
- PDN fixes
- timing fixes
- DRC fixes
- connectivity fixes

This guide was built from a fully successful 16-bit ALU implementation flow.

The goal is to provide a reusable backend implementation workflow for future ASIC projects.

COMPLETE ASIC FLOW OVERVIEW

```
RTL Design
↓
Simulation
↓
Synthesis
↓
Constraint Definition
↓
MMMC Setup
↓
Floorplanning
↓
```

```
Placement
↓
IO Pin Placement
↓
Clock Tree Synthesis (CTS)
↓
PDN Creation
↓
Detailed Routing
↓
DRC Verification
↓
Connectivity Verification
↓
Post-Route Timing
↓
GDSII Export
```

---

1. PROJECT DIRECTORY SETUP

Recommended Directory Structure

```
project/
│
├── RTL/
├── Constraints/
├── Scripts/
├── Synthesis/
├── Innovus/
├── Reports/
├── GDSII/
└── Screenshots/
```

---

2. RTL DESIGN STAGE

Required Files

```
ALU.v
```

```
tb_ALU.v
```

---

3. FUNCTIONAL SIMULATION

XCELIUM COMMANDS

Compile RTL + Testbench

```
xrun ALU.v tb_ALU.v
```

Generate Waveforms

```
xrun -access +rwc ALU.v tb_ALU.v
```

Common Problems

No Waveform Generated

Symptoms

- waveform viewer empty
- no VCD
- no signal activity

Causes

- missing dumpvars
- simulation ends too early
- missing access permissions

Fix

Use:

```
xrun -access +rwc
```

Add waveform dumping inside testbench.

---

4. SYNTHESIS STAGE

TOOL

Cadence Genus

START GENUS

```
genus
```

IMPORTANT

Never run TCL scripts from Linux shell directly.

WRONG:

```
source synth_clean.tcl
```

This causes:

```
bash: read_hdl: command not found
```

CORRECT METHOD

Inside Genus:

```
source Scripts/synth_clean.tcl
```

WORKING SYNTHESIS TCL SCRIPT

synth_clean.tcl

```
read_libs /opt/cadence/FOUNDRY/lan/flow/t1u1/reference_libs/GPDK045/
gsclib045_svt_v4.4/gsclib045/timing/slow_vdd1v0_basicCells.lib

read_hdl ../RTL/ALU.v

elaborate ALU

read_sdc ../Constraints/constraint.sdc

synthesize -to_mapped

report_area > area.rpt

report_power > power.rpt

report_timing > timing.rpt

write_hdl > ALU_16_synth.v

write_db design_ALU.db
```

IMPORTANT LIBRARY LESSON

GPDK45 vs GPDK90

Initial GPDK45 libraries produced:

```
Improper look-up table template
Missing axis name
```

FIX

Use compatible library:

```
slow_vdd1v0_basicCells.lib
```

COMMON SYNTHESIS ERRORS

ERROR:

```
Cannot use read_libs after read_mmmc
```

Cause

Library already loaded.

Fix

Restart Genus session.

ERROR:

```
No target technology library loaded
```

Cause

Missing read_libs.

Fix

Add:

```
read_libs <library>
```

before elaborate.

ERROR:

```
Cannot open file ../RTL/ALU.v
```

Cause

Wrong directory path.

Fix

Verify:

```
pwd
ls
```

Correct relative paths.

---

5. SDC CONSTRAINTS

WORKING SDC TEMPLATE

```
create_clock -name clk -period 10 [get_ports clk]

set_input_delay 2 -clock clk [all_inputs]

set_output_delay 2 -clock clk [all_outputs]

set_clock_uncertainty 0.1 [get_clocks clk]

set_load 0.1 [all_outputs]
```

IMPORTANT LESSON

Without clock constraints:

- WNS not generated
- TNS not generated
- unconstrained paths appear

ERROR:

```
Sequential clock pins without clock waveform
```

Cause

Clock not defined in SDC.

Fix

Use:

```
create_clock
```

ERROR:

```
Inputs without external driver/transition
```

Cause

No input delays.

Fix

Use:

```
set_input_delay
```

ERROR:

```
Outputs without external load
```

Cause

No output load definition.

Fix

Use:

```
set_load
```

---

6. SYNTHESIS REPORT CHECKING

IMPORTANT REPORTS

Area

```
report_area
```

Power

```
report_power
```

Timing

```
report_timing
```

Timing Intent

```
check_timing_intent
```

INTERPRETING TIMING

WNS

Worst Negative Slack

Closer to zero = better.

TNS

Total Negative Slack.

Lower magnitude = better.

---

7. INNOVUS STARTUP

START INNOVUS

```
innovus &
```

IMPORTANT LESSON

Never restore multiple designs in same Innovus session.

ERROR:

```
restoreDesign more than once dangerous
```

Fix

Always:

```
exit
```

Restart Innovus.

---

8. MMMC SETUP

WORKING innovus_mmmc.tcl

```
create_library_set -name slow_lib
-timing [list slow_vdd1v0_basicCells.lib]

create_rc_corner -name rc_corner

create_delay_corner -name delay_corner
-library_set slow_lib
-rc_corner rc_corner

create_constraint_mode -name constraints
-sdc_files [list constraint.sdc]

create_analysis_view -name setup_view
-constraint_mode constraints
-delay_corner delay_corner

set_analysis_view -setup [list setup_view]
-hold [list setup_view]
```

ERROR:

```
set_analysis_view not found
```

Cause

Incomplete MMMC setup.

Fix

Add:

```
set_analysis_view
```

ERROR:

```
cannot open SDC file
```

Cause

Wrong path.

Fix

Use correct relative path.

---

9. FLOORPLANNING

INITIALIZE DESIGN

```
init_design
```

CREATE FLOORPLAN

```
floorPlan -site core -r 1 0.7 20 20 20 20
```

CHECK FLOORPLAN

Die Area

```
dbGet top.fPlan.box
```

Core Area

```
dbGet top.fPlan.coreBox
```

COMMON FLOORPLAN PROBLEMS

ERROR:

```
Attribute specified in middle of DB hierarchy
```

Cause

Wrong dbGet syntax.

Fix

Use:

```
dbGet top.fPlan.box
```

NOT:

```
dbGet top.fPlan.box.size
```

---

10. STANDARD CELL PLACEMENT

PLACE DESIGN

```
placeDesign
```

CHECK CONGESTION

```
reportCongestion -overflow
```

IMPORTANT LESSON

Goal:

```
0% overflow
```

---

11. IO PIN PLANNING

ENABLE BATCH MODE

```
setPinAssignMode -pinEditInBatch true
```

A BUS PLACEMENT

```
editPin -fixOverlap 1 -unit MICRON -side Left -layer M4
-spreadType CENTER -spacing 0.6
-pin {A[0] A[1] A[2] A[3] A[4] A[5] A[6] A[7]
A[8] A[9] A[10] A[11] A[12] A[13] A[14] A[15]}
```

B BUS PLACEMENT

```
editPin -fixOverlap 1 -unit MICRON -side Left -layer M4
-spreadType CENTER -spacing 0.6
-pin {B[0] B[1] B[2] B[3] B[4] B[5] B[6] B[7]
B[8] B[9] B[10] B[11] B[12] B[13] B[14] B[15]}
```

RESULT BUS PLACEMENT

```
editPin -fixOverlap 1 -unit MICRON -side Right -layer M4
-spreadType CENTER -spacing 0.6
-pin {result[0] result[1] result[2] result[3] result[4]
result[5] result[6] result[7] result[8] result[9]
result[10] result[11] result[12] result[13] result[14] result[15]}
```

CONTROL SIGNALS

```
editPin -fixOverlap 1 -unit MICRON -side Top -layer M3
-spreadType CENTER -spacing 1.0
-pin {clk rst opcode[0] opcode[1] opcode[2] opcode[3] valid_in}
```

STATUS OUTPUTS

```
editPin -fixOverlap 1 -unit MICRON -side Bottom -layer M3
-spreadType CENTER -spacing 1.0
-pin {carry zero overflow negative valid_out}
```

DISABLE BATCH MODE

```
setPinAssignMode -pinEditInBatch false
```

IMPORTANT LESSON

Pins are NOT manually wired.

Routing connects them later automatically.

---

12. CLOCK TREE SYNTHESIS

CREATE CLOCK TREE SPEC

```
create_ccopt_clock_tree_spec
```

RUN CTS

```
ccopt_design
```

POST-CTS TIMING

```
timeDesign -postCTS
```

CLOCK TREE REPORT

```
report_ccopt_clock_trees
```

IMPORTANT LESSON

CTS improves:

- WNS

- TNS
- skew
- latency

---

13. PDN CREATION

IMPORTANT LESSON

PDN must be added BEFORE final routing.

CREATE POWER NETS

```
addNet VDD -power
```

```
addNet VSS -ground
```

CONNECT POWER NETS

```
globalNetConnect VDD -type pgpin -pin VDD -inst *
```

```
globalNetConnect VSS -type pgpin -pin VSS -inst *
```

APPLY GLOBAL NETS

```
applyGlobalNets
```

POWER RINGS

```
addRing
-type core_rings
-nets {VDD VSS}
-follow core
-layer {top Metal6 bottom Metal6 left Metal5 right Metal5}
-width 2
-spacing 1
-offset 2
```

POWER STRIPES

```
addStripe
-nets {VDD VSS}
-layer Metal4
-direction vertical
-width 1
-spacing 1
-set_to_set_distance 20
-start_offset 10
```

SPECIAL ROUTING

```
sroute
```

IMPORTANT PDN LESSONS

ERROR:

```
Global net VDD doesn't exist
```

Fix

Use:

```
addNet VDD -power
```

ERROR:

```
Polarity mismatch
```

Cause

Used generic nets instead of power/ground nets.

Fix

Use:

```
-power
-ground
```

ERROR:

```
-pitch illegal option
```

Cause

Older Innovus syntax.

Fix

Use:

```
-set_to_set_distance
```

instead of:

```
-pitch
```

---

14. DETAILED ROUTING

MAIN ROUTING COMMAND

```
routeDesign
```

IMPORTANT LESSON

Without routeDesign:

- connectivity violations appear
- nets remain open
- pins remain disconnected

ROUTING REPORTS

Congestion

```
reportCongestion -overflow
```

Timing

```
timeDesign -postRoute
```

ENABLE OCV MODE

```
setAnalysisMode -analysisType onChipVariation -cppr both
```

ERROR:

```
analysis mode needs to be set to OCV
```

Fix

Use:

```
setAnalysisMode -analysisType onChipVariation -cppr both
```

---

15. DRC VERIFICATION

RUN DRC

```
verify_drc
```

SAVE DRC REPORT

```
verify_drc -report drc_report.rpt
```

IMPORTANT LESSON

Initial DRC after PDN may fail.

Always rerun:

```
routeDesign
```

after PDN insertion.

---

16. CONNECTIVITY VERIFICATION

VERIFY CONNECTIVITY

```
verifyConnectivity -type all
```

IMPORTANT LESSON

Goal:

```
0 Viols
0 Wrngs
```

---

17. SAVE DESIGN CHECKPOINTS

SAVE FLOORPLAN

```
saveDesign floorplan.enc
```

SAVE CTS

```
saveDesign cts.enc
```

SAVE PDN VERSION

```
saveDesign pdn_complete.enc
```

SAVE FINAL DESIGN

```
saveDesign final_clean.enc
```

IMPORTANT LESSON

Always save checkpoints.

If Innovus crashes:

- restart
- restore previous checkpoint

RESTORE CHECKPOINT

```
restoreDesign cts.enc.dat ALU
```

---

18. FINAL GDSII EXPORT

STREAMOUT COMMAND

```
streamOut final_ALU_PDN.gds
-mapFile streamOut.map
-libName ALU
-structureName ALU
-unit 1000
-mode ALL
```

IMPORTANT LESSON

GDSII is:

- final chip layout
- fabrication database
- physical geometry export

---

19. FINAL EXPORTS

SAVE FINAL NETLIST

```
saveNetlist final_ALU.v
```

EXPORT SPEF

```
rcOut -spef final_ALU.spef
```

---

20. FINAL SUCCESS CHECKLIST

REQUIRED FINAL CONDITIONS

DRC

```
0 violations
```

Connectivity

```
0 violations
0 warnings
```

Routing

```
0 overflow
```

PDN

- rings created
- stripes created
- sroute successful

CTS

- completed successfully
- timing improved

GDSII

```
StreamOut finished successfully
```

---

21. MOST IMPORTANT ENGINEERING LESSONS

LESSON 1

Always constrain clocks correctly.

LESSON 2

Always save checkpoints.

LESSON 3

PDN must be inserted before final routing.

LESSON 4

Pins are NOT manually connected.

Routing connects them automatically.

LESSON 5

After PDN insertion:

Always rerun:

```
routeDesign
```

LESSON 6

Never panic at warnings immediately.

Distinguish:

- informational warnings
- critical errors
- flow-breaking issues

LESSON 7

Academic flows may still show:

- cap table warnings
- process node warnings
- via resistance warnings

These are usually non-fatal.

LESSON 8

Always verify:

- DRC
- connectivity
- timing

before GDS export.

---

22. FINAL RESULT

This workflow successfully achieved:

- RTL design
- synthesis
- timing constraints
- MMMC
- floorplanning
- placement
- CTS
- PDN implementation
- detailed routing
- DRC-clean layout
- clean connectivity
- post-route timing analysis
- final GDSII generation

This guide can now be reused for future ASIC backend implementation projects.

END OF GUIDE